ARCHITECTURE & VISUAL SPECIFICATION

青枢工具箱

架构设计与视觉规范

本文档完整定义青枢工具箱（QingShu.Toolkit）的工程架构与实现边界：全链路网络诊断引擎、磁盘清理策略、安全优化与功耗管理机制，以及基于 Fluent Design 的双主题视觉系统——青枢蓝 #2563EB 色板、Windows 10 亚克力材质与 WCAG AA 对比度约束，作为开发、评审与后续迭代的统一基准。

BeiXiangGuMao · QingShu.Toolkit 项目组

github.com/BeiXiangGuMao/QingShu.Toolkit

V1.0.0 · 2026 年 10 月 · WINUI 3 / .NET 8

目 录

第一章 项目概述与设计原则 2

1.1 项目定位与目标用户 2

1.2 三高设计目标 2

1.3 安全边界：明确不做的六件事 3

第二章 系统架构设计 3

2.1 分层架构 3

2.2 技术选型 4

2.3 依赖注入与扩展点 4

第三章 网络诊断引擎 5

3.1 分层检查模型 5

3.2 全链路 28 项检查清单 5

3.3 傻瓜模式修复流水线 6

3.4 日志留存体系 6

3.5 PDF 诊断报告模板 7

第四章 磁盘清理引擎 7

4.1 常规清理目标 7

4.2 微信与 QQ 久远文件（询问式） 8

4.3 大文件兜底扫描 8

第五章 性能优化与弹窗拦截 9

5.1 内存整理与进程 EcoQoS 9

5.2 全局白名单弹窗拦截 9

5.3 启动项只读与边界声明 9

第六章 功耗管理 10

6.1 状态机与自动切换 10

6.2 EcoQoS 实现细节 10

第七章 视觉规范 10

7.1 双主题色板（青枢蓝） 10

7.2 状态色与 WCAG AA 11
7.3 功耗标识色与使用纪律 11
7.4 材质策略（Win10 优先） 11
7.5 动效与组件 12
第八章 更新、部署与扩展 12
8.1 询问式自动更新 12
8.2 双模式打包与部署 12
8.3 数据与隐私 13

第一章 项目概述与设计原则

1.1 项目定位与目标用户

青枢工具箱（QingShu.Toolkit）是一套面向学校场景的 Windows 桌面系统工具箱，由 C# 与 WinUI 3 构建，运行于 .NET 8 LTS 与 Windows App SDK 1.6 之上，最低支持 Windows 10 1809（Build 17763）。它把校园终端最高频的四类痛点收敛到一个原生 Fluent 应用里：网络故障的排查与上报、磁盘空间的治理、系统性能的安全优化，以及功耗的自动化管理。命名采用「中文名 + 技术名」双轨制——产品名「青枢工具箱」，解决方案与主命名空间统一为 QingShu.Toolkit，其余模块按层级派生，保证代码库的可读性与检索性。

目标用户覆盖三类角色：任课教师需要「点一下就能用」的傻瓜式诊断与修复，不需要理解 DNS 与 DHCP 的区别；机房管理员与信息中心需要可留档、可追溯的诊断报告与日志体系，用于跨学期比对与向上报送；学生与公用终端则受益于弹窗拦截、清理与功耗策略带来的稳定与安静。三类角色共享同一套界面，通过功能分层与确认式交互来平衡易用性与安全性。

28	6	1
项网络检查（全链路分层）	个功能页面（仪表盘至设置）	次 UAC 合并提升（管理类修复）

1.2 三高设计目标

高性能

性能预算落实到具体机制上：所有探测、扫描与 WMI 查询一律进入后台线程，UI 线程只接收进度回调；常规清理与 IM 扫描采用流式枚举并设置条数与时长上限，避免大目录卡顿；实时监控采样间隔默认 2 秒，低功耗态自动放宽到 5 秒以上；应用本体在低功耗态主动应用 EcoQoS，把自身调度到能效核。发布形态为自包含单目录，冷启动不依赖额外运行时安装。

高颜值

视觉完全贴合原生 Fluent：Win10 使用 DesktopAcrylic 亚克力材质并按深浅主题调节不透明度，Win11 自动升级为 Mica；全套品牌色以「青枢蓝」为锚点定义浅色与深色两套主题字典，状态文字一律采用满足 WCAG AA 对比度的变体，而不是把亮色直接当文字色使用。动效克制——仅保留页面进入交错动画与原生控件过渡，确保低配机房依旧流畅。

高扩展

诊断检查项、清理目标、修复动作、弹窗白名单与 PDF 报告字段都是接口化的扩展点：新增一项检查只需实现 ICheck 并注册进编排器；新增一类清理目标只需描述路径与风险等级。服务全部通过依赖注入装配，视图模型与视图严格分离，为后续接入更多诊断能力或替换报告引擎留出空间。

1.3 安全边界：明确不做的六件事

工具箱的信任来自克制的边界。以下操作被产品层面明确禁止，并写进界面声明与代码注释，任何优化能力都不得触碰这些红线：

表 1-1 安全边界与替代方案

禁止操作	为什么不做	工具箱的替代做法
services.msc 改服务	服务配置影响面大，误改可导致系统异常	只读展示服务状态，给出 services.msc 手工处理指引
修改虚拟内存	页面文件策略因机而异，自动改动风险高	内存压力用工作集整理（可逆）缓解
导入注册表	.reg 批量导入难回滚、难审计	仅做注册表只读检查；个别设置通过系统 API 单点修改并记日志
NTLite 式精简	裁剪系统组件不可逆	不做任何组件裁剪
一键优化全家桶	黑盒批量改动无法解释	每个动作单列、可勾选、可解释、有输出
关闭 Defender	安全底线	不触碰安全软件，仅在防火墙状态检查中只读提示

第二章 系统架构设计

2.1 分层架构

整体采用「壳层 - 视图模型 - 服务 - 互操作」四层结构，依赖方向自上而下单向收敛。壳层由 MainWindow 承载自定义标题栏（含手动更新按钮）、NavigationView 导航与状态栏三色功耗标识；六个功能页面全部通过 x:Bind 绑定视图模型，视图模型使用 CommunityToolkit.Mvvm 的源生成特性实现属性与命令；服务层按领域拆分为网络诊断、清理、优化、功耗、弹窗、更新、报告与日志八组，全部以接口注册进 DI 容器；互操作层封装 Win32、WMI 与注册表只读访问，并对按需 UAC 提供统一的脚本化通道。

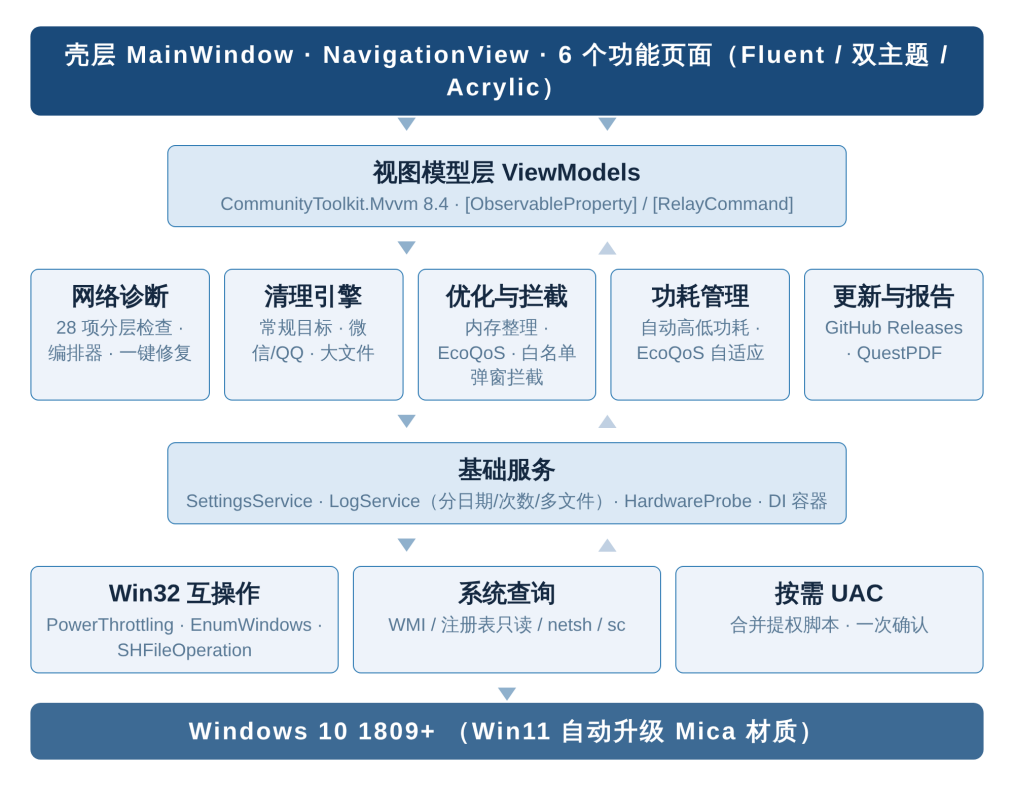


图 2-1 系统分层架构（自上而下单向依赖）

2.2 技术选型

表 2-1 关键技术选型

组件	选型	版本	选择理由
运行时	.NET 8 LTS	8.0	长期支持，Win10 1809 全兼容，校园老机友好
UI 框架	WinUI 3 / Windows App SDK	1.6.250602001	原生 Fluent，DesktopAcrylic 支持完善
MVVM	CommunityToolkit.Mvvm	8.4.2	源生成零反射，属性/命令样板最小化
DI	Microsoft.Extensions.DependencyInjection	8.0.1	标准生态，服务可插拔
PDF 报告	QuestPDF	2026.9.1	代码式布局，中文微软雅黑渲染，模板精致
硬件探测	System.Management（WMI）	8.0.0	序列号/驱动/网卡的标准通道
性能计数	System.Diagnostics.PerformanceCounter	8.0.0	CPU 占用采样

2.3 依赖注入与扩展点

应用启动时在 App.OnLaunched 中完成服务装配：配置、日志、硬件探测、功耗、弹窗、更新为单例；六个页面视图模型为瞬态，页面构造函数从 App.Services 解析。接口即边界——例如网络诊断编排器只暴露 CreatePlan 与 ExecuteAsync 两个方法，新增检查实现后无需改动任何上层代码。下表列出预留的扩展点与对应接入方式：

表 2-2 扩展点清单

扩展点	接入方式	典型用途
网络检查项	实现 ICheck，注册进编排器检查数组	加入校园认证厂商私有接口探测
清理目标	在 CleanupService 目标表中追加 CleanTargetDef	接入机房教学软件缓存目录
修复动作	在 NetworkRepairService 中注册命令映射	增加隧道/虚拟网卡重置
弹窗拦截策略	设置中的白名单与规则表	按校区下发白名单配置
报告模板	替换 PdfReportService 组合函数	加入省级上报格式

第三章 网络诊断引擎

3.1 分层检查模型

网络故障的根因分布在不同层级，逐层收敛是唯一定位思路。引擎把 28 个检查项组织为十二个分层，从物理链路开始逐级向上：物理与硬件不通时，上层结论自动降级为「先修下层」；各层之间通过共享上下文 CheckContext 传递快照（主网卡、网关、DNS 列表、Wi-Fi 信息）与跨层结论（例如外网 HTTP 是否放行，直接决定城域网认证过期的判定方式）。全部检查并行受控于统一的取消令牌，任一环节可中断。

每个检查项输出统一的状态机：通过、警告、未通过、已修复、已跳过，并附带结果说明、可执行的处置建议与详细数据行。状态直接绑定 UI 的状态徽标，同时写入会话日志与 PDF 报告，保证「屏幕上看到的」与「报告里写的」完全一致。

3.2 全链路 28 项检查清单

表 3-1 检查项清单（按分层）

分层	检查项	关键判据（摘要）
物理链路	物理连接状态；Wi-Fi 信号质量	OperationalStatus；链路速率；信号百分比阈值 65%/40%
硬件驱动	网卡 PnP 与驱动	ConfigManagerErrorCode 28/22/10/31/43；驱动版本
IP 配置	IP 与掩码；DHCP 租约	APIPA 169.254 识别；租约到期时间（WMI）
网关路由	网关连通；ARP 表项；默认路由	丢包/延迟阈值；ARP 缺失与同 MAC 多 IP；0.0.0.0/0 存在性
DNS	配置；系统解析；逐台探测；Hosts 劫持	无效 DNS 地址；UDP 53 原生查询；关键域名映射
互联网	ICMP；HTTP（NCSI）；HTTPS/TLS	多公共 IP 对照；connecttest 预期体；证书错误归类
校园内网	内网资源探测	设置中的内网地址 ping 与 TCP 80 兜底
城域网认证	门户检测；过期判定；被踢识别	NCSI 重定向；状态接口关键字；门户可达但外网不通
系统时钟	时钟偏差	HTTP Date 对照，60/300 秒两级阈值
代理与 VPN	用户代理；WinHTTP 代理；VPN 干扰	ProxyEnable/PAC；winhttp 状态；隧道适配器
系统网络栈	服务状态（只读）；防火墙；Winsock	关键服务运行态；配置文件开关；第三方 LSP
连接质量	延迟抖动；MTU	10 次 ping 统计；DF 探测 1500/1492/1480

3.3 傻瓜模式修复流水线

诊断结束后自动生成修复清单，每个动作都是一行可读的人话——做什么、为什么、要不要管理员、有没有风险。普通动作（刷新 DNS、关闭用户代理、打开认证页）在本进程内直接执行；管理类动作（ARP 清理、DHCP 续租、网卡启停、时间同步、Winsock 重置、DNS 改指公共）被合并为一份批处理脚本，经一次 UAC 提权统一执行，脚本逐行输出成功与失败标记，执行完毕回填到界面与日志。危险项（Winsock 重置需重启、DNS 改指公共）默认不勾选，必须用户显式确认。

> **为什么是一次 UAC：**传统工具逐条弹窗极易造成「无脑点确定」。合并提权把管理类动作打包为一份可审计的临时脚本（执行前写入日志，执行后保留输出），既减少打扰，也让整个提升过程可回溯。

3.4 日志留存体系

日志按「日期 - 当日次数 - 多文件」三维度组织，诊断会话每次生成成对的可读文本与结构化 JSONL 文件，应用运行日志按月归档，清理行为单独落 JSON 快照，默认保留 90 天且启动时自动清理过期目录：

```
%LOCALAPPDATA%\QingShu.Toolkit\logs\
app-202610.log / app-202610.jsonl （应用运行日志，按月）
2026-10-24\diag-20261024-143052-003.log （第 3 次诊断·可读版）
2026-10-24\diag-20261024-143052-003.jsonl （第 3 次诊断·结构化版）
2026-10-24\cleanup-20261024-150010.json （清理报告快照）
```

内置日志查看页支持按日期浏览、文件选择与尾部预览，管理员也可以直接打开文件夹拷贝归档。JSONL 每行携带时间戳、级别、类别、消息与附加数据，便于后续接入脚本化分析或批量比对。

3.5 PDF 诊断报告模板

当自动修复无法覆盖问题（例如硬件故障、出口策略），教师需要一份能直接交给信息中心的正式材料。报告由 QuestPDF 按固定模板生成：页眉为品牌色与报告编号（QS-DIAG-yyyyMMdd-NNN，按日累计次数），正文依次为设备与位置信息、结论概览、分层诊断明细、修复记录、分级处置建议与确认签署栏。

表 3-2 报告字段与来源

字段	来源	说明
报告编号	生成时按当日报告数自增	QS-DIAG-20261024-003 形式
设备编号	导出对话框填写，默认主机名	可与学校资产编号对应
序列号 / MachineGuid	WMI Win32_BIOS / 注册表只读	设备唯一性双保险
年级 / 班级 / 报告人	导出对话框必填校验	定位教室与责任人
诊断明细	会话结果集	状态着色，失败行浅红底
处置建议	由结果自动推导 P1/P2/P3	并给出升级路径与建议对象

报告全程本地生成，文件落盘到「文档\QingShu.Toolkit\Reports\日期」目录，文件名包含设备编号与时间戳，便于按机归档。中文渲染依赖系统微软雅黑字体，Win10 中文环境默认可用，无需额外安装字体。

第四章 磁盘清理引擎

4.1 常规清理目标

常规清理遵循「白名单目录 + 风险分级 + 逐文件容错」三原则：只清理已知安全的目录，每个目标标注风险等级与是否需要管理员，删除时逐文件 try/catch，被占用文件计入跳过而不是中断。扫描先给出体积统计，用户勾选确认后才会执行删除，删除方式默认进回收站：

表 4-1 常规清理目标

目标	风险	管理员	说明
用户 / 系统临时文件	低	系统侧需要	标准 TEMP 目录内容清理
Windows 更新下载缓存	中	是	SoftwareDistribution\Download；安装中文件自动跳过
传递优化缓存	中	是	P2P 分发缓存
缩略图 / 图标缓存	低	否	首次打开文件夹重建，属正常现象
Edge / Chrome / Firefox 缓存	低	否	仅 Cache 类目录，不动收藏与密码
错误报告 / 崩溃转储	低-中	部分	WER 队列与 dump 文件
DirectX 着色器缓存	低	否	D3DSCache
Office 文档缓存	中	否	默认不勾选，云端编辑中请勿清理
Windows.old	高	是	默认不勾选，删除后无法回退旧系统
回收站	中	否	Shell API 清空，显示体积与条目

4.2 微信与 QQ 久远文件（询问式）

即时通讯工具是教室电脑磁盘占用的隐形大头。引擎兼容微信 3.x（WeChat Files）、微信 4.x（xwechat_files）与 QQ 传统及 nt_qq 新版布局，按账号目录遍历，仅列出超过阈值天数（默认 180 天，可配置）的媒体与文件缓存，按类型与账号分组展示体积与最后修改时间。整个过程是「询问式」的：扫描只读、结果全选可取消、删除方式跟随全局设置（默认回收站），界面明确提示删除久远媒体后对应历史聊天图片将显示为过期。

> **聊天记录数据库受保护：**所有 .db、.sqlite 及 config、backup 目录被硬编码排除在扫描与删除范围之外，任何操作都不会触碰聊天记录本身；若用户修改过微信/QQ 存储位置，可用大文件扫描作为兜底手段。

4.3 大文件兜底扫描

对于不认识路径来源的空间占用，大文件扫描提供全盘兜底：枚举固定分区，跳过 Windows、Program Files、ProgramData 与回收站等系统目录，按阈值（默认 100MB）收集并按体积倒序展示前 200 项，支持打开所在位置核对后再删除。扫描设置 120 秒时长上限与 800 条数量上限，避免极端磁盘拖垮体验。

第五章 性能优化与弹窗拦截

5.1 内存整理与进程 EcoQoS

内存整理基于 EmptyWorkingSet：把所选进程的物理工作集页回收进系统待用列表，进程继续正常运行，等价于任务管理器的「内存压缩」思路，不做任何不可逆动作。界面默认列出占用最高的 25 个进程并排除系统关键进程；执行后统计实际释放量与跳过数。进程级 EcoQoS 则通过 SetProcessInformation 的功耗遏制接口，把后台闲置进程调度到能效档位，适合更新、同步类任务；操作可一键恢复，状态在列表中实时标注。

5.2 全局白名单弹窗拦截

弹窗拦截采用全局白名单策略（按需求选定的最激进档）：只放行白名单进程的窗口，其余新出现的可见窗口一律被动关闭。「被动」意味着不做注入、不挂钩 DLL，仅通过窗口枚举与 PostMessage 关闭消息实现。为了把误伤概率压到最低，引擎内置五重护栏：

1. 启动快照：服务启动时已存在的窗口一律视为基线，永不拦截；

2. 前台豁免：用户正在交互的前台窗口绝不关闭；

3. 外壳豁免：任务栏、桌面、输入窗口等系统外壳类按类名放行；

4. 工具窗口豁免：WS_EX_TOOLWINDOW 属性窗口（悬浮条、提示层）不拦截；

5. 白名单兜底：系统、浏览器、办公与教学软件（希沃、WPS、Office、QQ、微信等）默认放行，可视化增删。

每次拦截都会把窗口标题、类名与进程名写入日志，状态栏实时累计当日次数；关闭拦截只需一个开关，重启应用后仍以设置为准。该策略的适用前提是教学机软件环境相对固定，若校区环境复杂，建议先在观察模式（仅看日志不开启）运行一周再扩充白名单。

5.3 启动项只读与边界声明

启动项采用只读设计：展示 HKCU/HKLM Run 键与启动文件夹中的条目及来源，不提供任何禁用开关，一键跳转系统任务管理器处理。这与「不做一键优化全家桶」的边界一致——展示信息、解释现状、把决定权与操作权交还给管理员，工具只负责让信息可见。

第六章 功耗管理

6.1 状态机与自动切换

功耗模块围绕三态标识与四档模式构建。三态是界面上的小圆点颜色：绿色低功耗、蓝色平衡、红色高性能，只做小面积标识，禁止大面积填充；四档是用户可选模式：自动、平衡、高性能、低功耗。自动模式下按电源状态实时切换——接通电源进入高性能，使用电池自动降为低功耗并对本程序应用 EcoQoS。状态来源为 GetSystemPowerStatus 的 10 秒轮询兜底，可靠且不依赖系统事件订阅。

表 6-1 模式与标识色

模式	触发条件	标识色（浅色 / 深色）	EcoQoS
自动·高性能	自动模式且已接通电源	#DC2626 / #F87171	关闭
自动·低功耗	自动模式且使用电池	#059669 / #34D399	本程序开启
平衡	手动选择	主色 #2563EB / #3B82F6	关闭
手动高性能 / 低功耗	手动选择	对应标识色	跟随选择

6.2 EcoQoS 实现细节

EcoQoS（Economic Quality of Service）通过 kernelbase 的 SetProcessInformation 接口下发 PROCESS_POWER_THROTTLING_EXECUTION_SPEED 控制码实现：开启后系统把进程调度到能效核或降频档，关闭即恢复。该机制自 Windows 10 1709 起可用，完全覆盖目标系统区间。对本程序的开关由设置项控制，对第三方进程的操作仅发生在性能优化页且由用户显式选择，权限不足的进程会被跳过并计数。

第七章 视觉规范

7.1 双主题色板（青枢蓝）

品牌主色为青枢蓝：浅色主题 #2563EB，深色主题提亮为 #3B82F6 以免发灰。全部颜色以主题字典形式定义，控件随主题自动切换；同时覆写 SystemAccentColor 系列，让按钮、滑块等原生控件同步品牌化。

表 7-1 中性色（浅色 / 深色）

用途	浅色	深色	XAML Key
页面背景	#F8FAFC	#0F172A	PageBackgroundBrush
卡片背景	#FFFFFF	#1E293B	CardBackgroundBrush
边框 / 分割线	#E2E8F0	#334155	CardBorderBrush
一级正文	#1E293B	#F1F5F9	TextPrimaryBrush
二级正文	#64748B	#94A3B8	TextSecondaryBrush
占位文字	#94A3B8	#64748B	PlaceholderTextBrush
Hover / Pressed	#EFF6FF / #DBEAFE	#1E3A8A / #1E40AF	HoverFillBrush / PressedFillBrush

深色模式不使用纯黑，而以 #0F172A 深炭蓝为基底，与亚克力材质融合自然，长时间观看不易疲劳；卡片以 #1E293B 上浮一层形成层次。两套背景同时提供半透明变体 PageBackgroundAcrylicBrush，在开启材质时让亚克力从页面底部透出。

7.2 状态色与 WCAG AA

状态色采用「填充原色 + 文字深化变体」的双轨逻辑：图形填充使用品牌状态色原色，而状态文字一律使用对比度达标的变体（浅色主题加深、深色主题提亮），浅底 Chip 使用 10%～15% 透明填充。以浅色主题为例：成功文字 #047857、警告文字 #92400E、错误文字 #B91C1C、信息文字 #1D4ED8，在 #FFFFFF 与 #F8FAFC 背景上均满足 WCAG AA 的 4.5:1 要求；深色主题对应 #6EE7B7、#FCD34D、#FCA5A5、#93C5FD。

> **黄色使用纪律：**#F59E0B 在白底上对比度不足，永远不作为正文文字色出现；作为填充时也仅用于图标与细小元素，正文语境一律使用深化变体。

7.3 功耗标识色与使用纪律

功耗标识是本项目特色：绿色（低功耗）、主色（平衡）、暗红（高性能）三色只允许出现在状态栏小圆点、功耗卡片的小徽标与模式单选的着色点上，任何页面都不得用这些颜色做大面积背景或整行填充，避免在课堂教学场景产生视觉干扰。深色主题下的三色分别提亮为 #34D399、#3B82F6 与 #F87171。

7.4 材质策略（Win10 优先）

表 7-2 窗口材质参数

平台	材质	参数
Windows 11（22000+）	Mica	系统自动适配主题，无需额外参数
Windows 10	DesktopAcrylic	浅色：Tint #F8FAFC，不透明度 0.72；深色：Tint #0F172A，不透明度 0.85
不支持材质	回退不透明	页面使用 PageBackgroundBrush 纯色

深色亚克力刻意提高不透明度（更「实」），是为了在深色底上保持文字对比度不受背景杂色影响；材质可通过设置一键关闭，关闭后立即回到纯色页面。实现集中在 WindowBackdropService，含激活态联动、主题切换重建与资源释放。

7.5 动效与组件

动效克制而完整：页面进入使用 EntranceThemeTransition（自下 24 像素、交错启用），页间导航使用系统 EntranceNavigationTransitionInfo，其余交互动效全部沿用 Fluent 原生实现，不自绘装饰动画。组件层面统一约定：卡片为 8 像素圆角、1 像素描边、16 像素内边距；状态徽标为胶囊形浅底；对话框使用原生 ContentDialog，危险操作前置确认并明示后果。字体遵循系统字型栈（中文自动落到微软雅黑），路径与日志使用 Consolas 等宽。

第八章 更新、部署与扩展

8.1 询问式自动更新

更新基于 GitHub Releases，全程询问式、无静默安装：应用启动后按 24 小时间隔检查（可关闭），发现新版本先弹窗询问；用户同意后开始下载并展示进度；下载完成再次确认，随后生成 update.cmd——等待主进程退出、解压新包、robocopy 覆盖、清理临时目录并重启应用。标题栏右上角（最小化按钮左侧）提供手动更新按钮，带红点提示待更新状态。校园网无法访问 GitHub 时可离线替换安装包，不影响其他功能。

1. 后台检查 GitHub Releases 最新版本，按语义化版本比较；
2. 发现新版本 → 对话框询问是否下载；
3. 下载完成 → 再次确认是否立即安装；
4. 确认后生成更新脚本并退出主程序；
5. 脚本完成替换与清理后自动重启应用。

8.2 双模式打包与部署

默认构建为未打包自包含单目录：dotnet publish 携带 .NET 8 与 Windows App SDK 运行时，U 盘拷贝或局域网共享即可在任意 Win10 1809+ 机器运行，免安装、免管理员，最适合机房

批量部署。项目同时保留 MSIX 模式：切换 WindowsPackageType 并启用 packaging/Package.appxmanifest 即可产出 商店式安装包。两种模式共享同一套代码与配置，按部署场景选择。

推荐的下发流程为：管理机发布 → 共享目录 → 机房批处理 robocopy → 可选预写 config.json（预置城域网认证地址、内网探测地址与白名单），使首次开箱即可执行有效诊断。

8.3 数据与隐私

表 8-1 本地数据位置

数据	位置	说明
应用配置	%LOCALAPPDATA%\QingShu.Toolkit\config.json	JSON 可读，可直接下发预置
诊断 / 运行日志	%LOCALAPPDATA%\QingShu.Toolkit\logs\	按日期与次数分文件，默认保留 90 天
PDF 报告	文档\QingShu.Toolkit\Reports\日期\	按日编号，含设备与位置信息
更新缓存	%LOCALAPPDATA%\QingShu.Toolkit\update\	更新包与 update.cmd

所有诊断、清理与报告数据仅在本机处理与留存，不上传任何数据；唯一联网行为是可选的更新检查。崩溃信息写入本地 crash.log 供排查，不包含任何用户内容。该隐私边界与安全边界一样，是产品对校园场景的长期承诺。